HOME MAINTENANCE REQUEST HUB

TABLE OF CONTENTS

1. TITLE **1**

2. INTRODUCTION & OBJECTIVES **3**

3. TOOLS/D\ENVIRONMENT USED……...5

4. ANALYSIS DOCUMENT **7**

a. SRS……...8
b. DATA FLOW DIAGRAM 10
c. ENTITY RELATIONSHIP DIAGRAM……15

5. DESIGN DOCUMENT **16**
a. MODULES WITH DESCRIPTION WITH EFFORT ESTIMATION 16
b. DATA STRUCTURE FOR ALL MODULES..... 17
c. DATABASE DESIGN……...18
d. DATA INTEGRITY AND CONSTRAINTS……...21
e. MODULES WITH PROCESS LOGIC……..22
f. PROCEDURAL DESIGN…….23
g. USER INTERFACE DESIGN……..25
6.PROGRAM CODE…….26

6. TESTING DETAILS……..34
a. REPORT GENERATION …….35
7. INPUT AND OUTPUT SCREENS……42
8. IMPLEMENTATION OF SECURITY……..46
9. LIMITATIONS OF THE PROJRCT……..48
10. PROJECT FOR ANY INDUSTRY/CLIENT: NO
11. FUTURE APPLICATIONS…….49
12. BIBLIOGRAPHY…….50

INTRODUCTION

In today's busy world, maintaining a home can be both time-consuming and challenging, making reliable and efficient home repair services more essential than ever. **Home Maintenance Request Hub** aims to simplify home maintenance by connecting homeowners and tenants with trusted service providers through a streamlined, web-based platform. Home Maintenance Hub allows users to submit repair requests, schedule services, and track the progress of their maintenance tasks—all in one place. Whether it's plumbing, electrical work, or routine maintenance, the platform promotes hassle-free home care while ensuring quality and timely service. With features such as real-time status updates, feedback systems, and seamless payments, **Home Maintenance Request Hub** is designed to make home upkeep easier, more efficient, and more reliable for everyone involved.

In today’s fast-paced and demanding lifestyles, home maintenance is often overlooked or delayed due to the lack of time, trustworthy service providers, or efficient scheduling systems. With urban living spaces increasing in complexity and modern households relying heavily on a range of technical services, the demand for reliable, responsive, and quality home maintenance has never been greater. Traditional methods of managing home repairs—through manual scheduling, unverified contractors, or phone-based service requests—are often inefficient, lack transparency, and leave room for miscommunication or substandard service delivery.

The Home Maintenance Request Hub is designed to address these challenges by introducing a centralized, digital platform where homeowners and tenants can effortlessly manage their maintenance needs. Whether it’s a leaking faucet, electrical faults, HVAC issues, or routine servicing, the platform bridges the gap between customers and certified service providers, streamlining the entire process from request initiation to task completion.

This web-based system offers a robust suite of features that make it easy for users to register, submit service requests, track progress in real-time, and provide feedback post-service. By integrating technologies like real-time notifications, secure payment gateways, and profile management tools, the platform not only ensures service quality and accountability but also enhances the user experience by making the process more predictable and convenient.

Furthermore, the platform emphasizes trust and professionalism by incorporating a stringent

verification system for service providers, enabling users to make informed choices based on ratings, experience, and customer reviews. For administrators, the system provides oversight tools to manage users, monitor performance, and resolve disputes efficiently.

The Home Maintenance Request Hub is not merely a convenience tool—it is a complete service ecosystem aiming to redefine how individuals approach home care. It combines usability, reliability, and security to foster an environment where timely, high-quality home maintenance becomes the norm rather than the exception.

OBJECTIVES

The primary objective of **Home Maintenance Request Hub** is to create an intuitive, user-friendly system that connects homeowners, tenants, and service providers, simplifying the process of managing home repairs and maintenance tasks. By offering a seamless platform for submitting and handling service requests, managing profiles, and ensuring smooth communication, the platform aims to make home maintenance more efficient, reliable, and convenient for all users. Ultimately, the project seeks to enhance the home repair experience by fostering trust, transparency, and timely service, making home care hassle-free.

- **Streamlined Request Management and Communication**: To facilitate clear and efficient communication between homeowners and service providers, enabling users to su maintenance requests easily, receive updates, and schedule services with confidence.
- **User-Friendly Interface**: To develop a simple, accessible web platform where homeowners and tenants can easily navigate the system, submit maintenance requests, track the progress of ongoing tasks, and manage their profiles with minimal effort.
- **Efficient Service Provider Registration and Management**: To create a comprehensive registration process for service providers, allowing them to submit essential information such as skills, certifications, and availability. The platform will include features for admin to verify provider credentials and approve applications, ensuring only qualified professionals offer services.

- **Maintenance Request Tracking System**: To implement a system that allows homeowners and tenants to track the status of their requests in real-time, view service provider availability, and receive updates on service progress, ensuring a smooth and reliable experience from request to completion.
- **Feedback and Rating Mechanism**: To establish a feedback system where both homeowners/tenants and service providers can leave reviews and ratings for each other, helping future users make informed decisions and promoting quality service based on past experiences.
- **Real-Time Status Updates and Notifications**: To provide homeowners and tenants with real-time notifications on request status, service provider arrival times, and other important information, ensuring transparency and timely home maintenance services.
- **Cost Efficiency and Service Quality**: To encourage the use of verified and professional service providers, ensuring high-quality maintenance services at competitive prices, ultimately reducing the hassle and cost of home care while maintaining high standards.

TOOLS AND ENVIRONMENT

Hardware Specifications

- Processor: Quad-core processor.
- RAM: Minimum of 8 GB.
- Storage: SSD with at least 256 GB.

Software Specification

1. Operating System

Primary OS Options:

Linux Distributions: Ubuntu or CentOS (preferred for production due to performance, security, and stability).

Windows: Suitable for development and testing environments with user-friendly GUI.

2. Database Management System (DBMS)

MySQL (Relational Database):

Chosen for its reliability, performance, and open-source availability.

Ensures efficient handling of structured data like user information, service requests, payments, an feedback.

Features like indexing and foreign key constraints enable fast querying and maintain referential Integrity.

3. Programming Languages and Frameworks.

Frontend:

HTML5: Structure and semantic layout of web pages.

CSS3: Styling and responsive design for user-friendly interfaces.

JavaScript: Client-side interactivity and asynchronous communication (e.g., using AJAX or Fetch API).

Backend:

Python: Core programming language for server-side logic.

Django Framework: A high-level, secure, and scalable Python web framework used to:

Implement MVC architecture.

Handle user authentication and authorization.

Manage service workflows, ORM-based database operations, and session management.

4. Development Tools

Visual Studio Code (VS Code):

Lightweight, highly customizable IDE with support for Python, HTML, CSS, and JavaScript.

Extensions for Django, debugging tools, Git integration, and database viewers enhance development efficiency.

Version Control:

Git: For tracking code changes and managing collaborative development.

GitHub or GitLab: For code hosting, issue tracking, and release management.

5. Additional Tools and Technologies (Optional Enhancements)

Postman: For testing REST APIs during backend development.

Docker: For containerizing the application to ensure consistent environments across development, testing, and production.

Nginx/Apache: For deploying the Django application in a production environment with load balancing and security.

Celery + Redis (for Advanced Use): For asynchronous task handling, such as sending notifications or emails

6. Security Considerations

Django Security Features:

Protection against CSRF, XSS, and SQL injection.

Secure user authentication and password hashing.

HTTPS Implementation: Ensures encrypted data transmission between client and server.

ANALYSIS DOCUMENT

(a) Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

The purpose of this document is to define the software requirements for the Home Maintenance Request Hub. This system aims to streamline the process of home maintenance by bridging thegap between service providers and homeowners/tenants using a web-based interface.

1.2 Scope

The system provides functionalities for homeowners to submit maintenance requests, for service providers to manage jobs, and for admins to oversee platform operations. It includes user authentication, request tracking, feedback, payment integration, and report generation.

1.3 Intended Audience

Software developers

UI/UX designers

Test engineers

1.4 Definitions, Acronyms, and Abbreviations

SRS: Software Requirements Specification

UI: User Interface

DBMS: Database Management System

DFD: Data Flow Diagram

2. Overall Description

2.1 Product Perspective

This is a stand-alone, full-stack web application built using Django (backend) and standard front-end technologies. It interfaces with a MySQL database and supports role-based access.

2.2 Product Functions

User registration and login (admin, homeowner/tenant, service provider)

Request creation and tracking

Real-time notifications

Feedback and rating

Payment processing

Report generation

2.3 User Characteristics

Homeowners/Tenants: Basic web usage knowledge; can submit requests and manage profiles.

Service Providers: Skilled workers who accept/manage job requests.

Admins: Technically proficient users who manage the platform.

2.4 Operating Environment

Client Side: Any modern browser (Chrome, Firefox, Edge)

Server Side: Django on Linux or Windows

Database: MySQL

3. **Specific Requirements**

3.1 Functional Requirements

1. User Registration and Authentication

Support role-based login.

Validate credentials and provide password encryption.

2. Maintenance Request Handling

Allow homeowners to create, edit, cancel, and track requests.

Assign requests to available providers automatically or manually.

3. Service Provider Module

View incoming job requests.

Accept, schedule, and update job progress.

4. Admin Dashboard

Approve/reject registrations.

Monitor feedback, track requests, and generate reports.

5. Feedback System

Two-way feedback mechanism between homeowners and providers.

Ratings from 1 to 5 stars with optional comments.

6. Notifications

Real-time alerts for job status, updates, and system messages.

7. Payments

Secure payment gateway integration (online/cash options).

Track transaction history.

8. Reports

Generate reports on service usage, payments, provider performance.

3.2 Non-Functional Requirements

Performance: System should support at least 100 concurrent users.

Scalability: Modular architecture to support future mobile or IoT integrations.

Security: Use of HTTPS, input validation, CSRF/XSS protection.

Usability: Responsive UI, minimal clicks for task completion.

Reliability: 99.9% uptime with database backup every 24 hours.

4. System Models

4.1 Use Case Diagram

[Can be attached visually; shows actors (Homeowner, Admin, Provider) interacting with major modules.]

4.2 Data Flow Diagrams

Level 0: Basic request and response loop

Level 1: Detailed flows for Admin and Service Provider (provided in your project)

4.3 Entity-Relationship Diagram

Covers entities: User, Service Request, Feedback, Payment, Notifications.

5. External Interfaces

5.1 User Interface

Clean, intuitive web forms for request entry, feedback, and dashboards.

5.2 Hardware Interfaces

Server with minimum 8GB RAM, SSD storage, and quad-core processor.

5.3 Software Interfaces

MySQL DB

Django ORM

REST APIs for internal module communication

(b) DATA FLOW DIAGRAMS

A Data Flow Diagram (DFD) is a visual representation used in software engineering to illustrate how data moves through a system. It focuses on the flow of information rather than on the logic or timing of processes. DFDs help in understanding how inputs are transformed into outputs through various components of a system. They are especially useful during the early stages of system analysis and design, as theyw provide a clear and structured way to communicate system functionality to both technical and non-technical stakeholders.

In a DFD, the key elements include processes, data stores, external entities, and data flows. A process represents an operation or function that transforms input data into output; this is usually depicted as a circle. Data stores are places where data is kept for future use and are shown as open-ended rectangles. External entities are people, systems, or organizations that interact with the system, typically shown as squares. Finally, data flows are arrows that indicate the direction and movement of data between processes, data stores, and external entities.

DFDs are typically developed in levels. A Level 0 DFD, also called a context diagram, shows a high-level overview of the entire system as a single process with interactions to external entities. Level 1 and higher DFDs break that main process into sub-processes, revealing more details about the system's internal operations.

Level 0 DFD (Context-Level Diagram)

This is the highest-level overview of the system. It includes:

Homeowner/Tenant: Users who raise service requests.

Service Provider: Professionals who respond to and handle the requests.

Admin: Manages and oversees system operations.

Main data flows:

Requests from Homeowners/Tenants go to the system.

Responses come back from the Service Provider.

Admin can manage or monitor these interactions.

Level 1 DFDs (More Detailed Breakdown)

Admin Module :

Login/Register: Admin logs in or registers.

Manage User Accounts: Admin can approve/reject registrations.

Manage Service Requests: Admin schedules and assigns service jobs.

View/Manage Feedback: Admin can monitor feedback to ensure quality control.

Service Provider Module :

Login/Register: Service provider logs in or registers.

Job Scheduling: Providers view and manage assigned jobs.

Track Requests: Track progress/status of tasks.

View Feedback: Access reviews and ratings.

Manage Profile: Update personal and professional details.

Homeowner/Tenant Module :

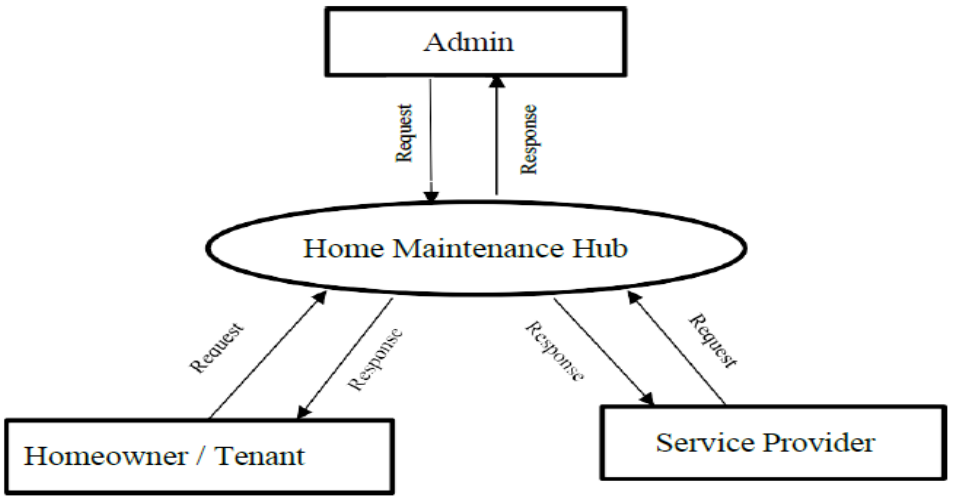
Submit Requests: Users raise maintenance requests.

Track Progress: Monitor request status.

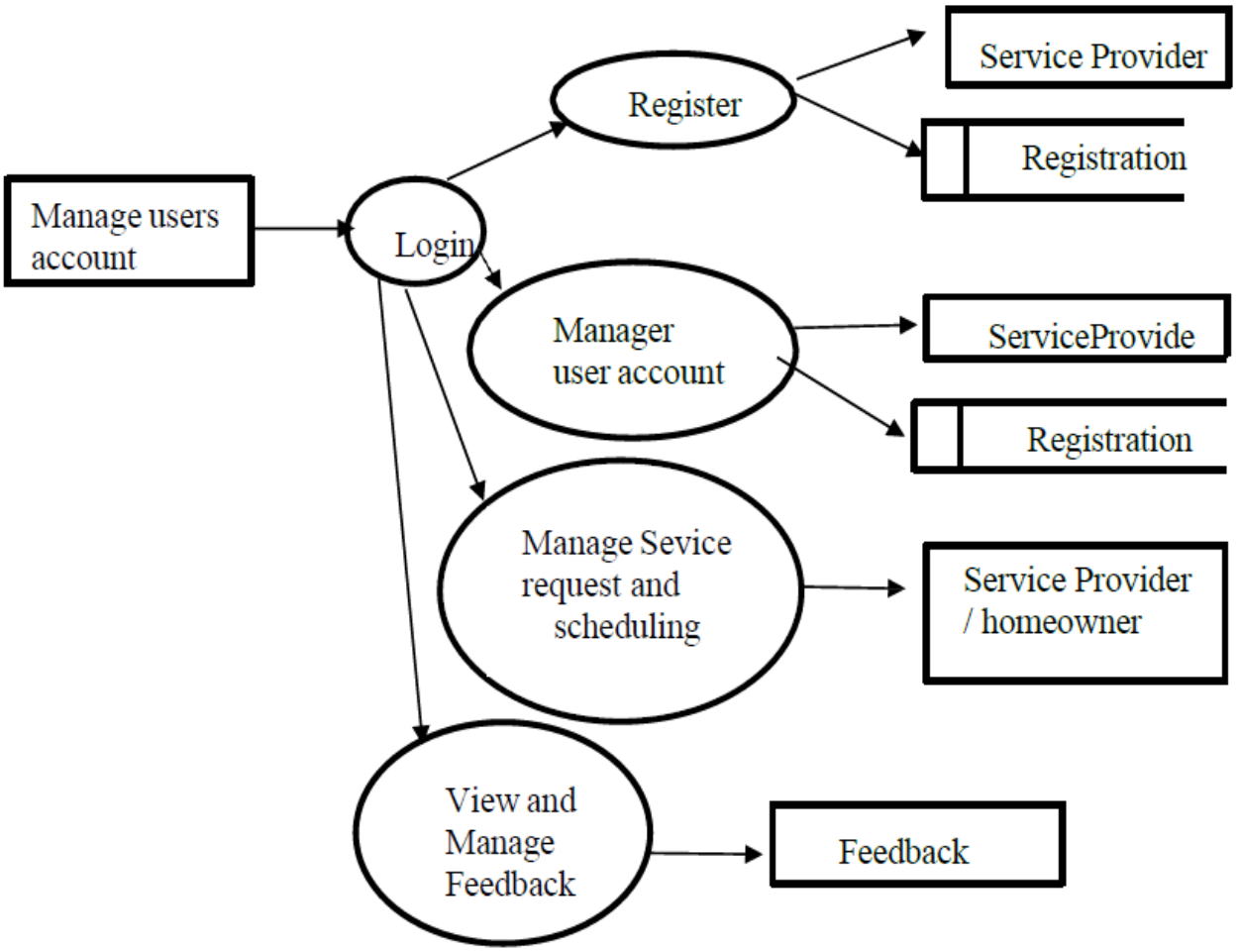
Search Providers: Look for qualified professionals.

Feedback: Provide reviews after task completion

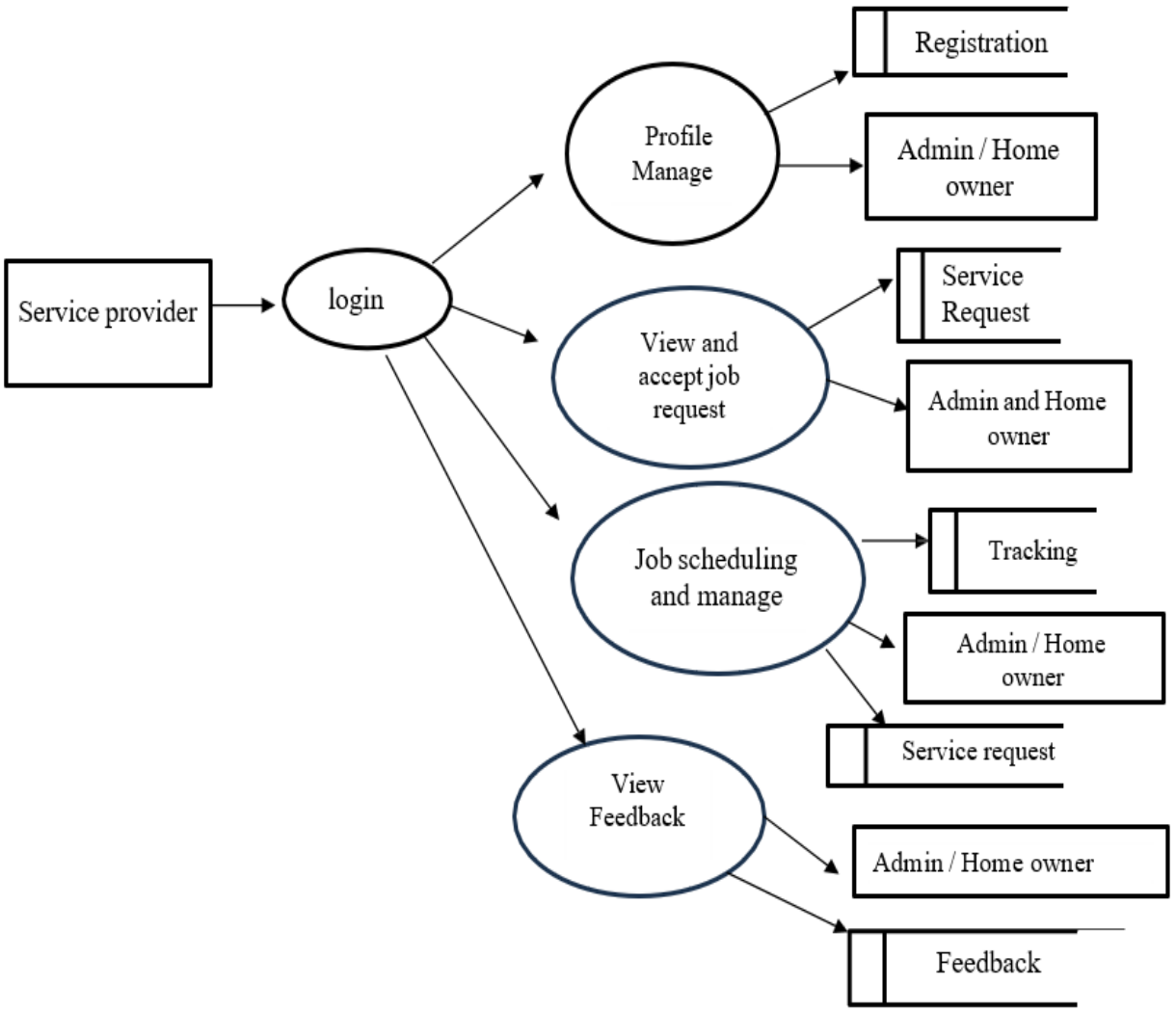
(a) DATA FLOW DIAGRAM : LEVEL 0



LEVEL 1 DFD ADMIN

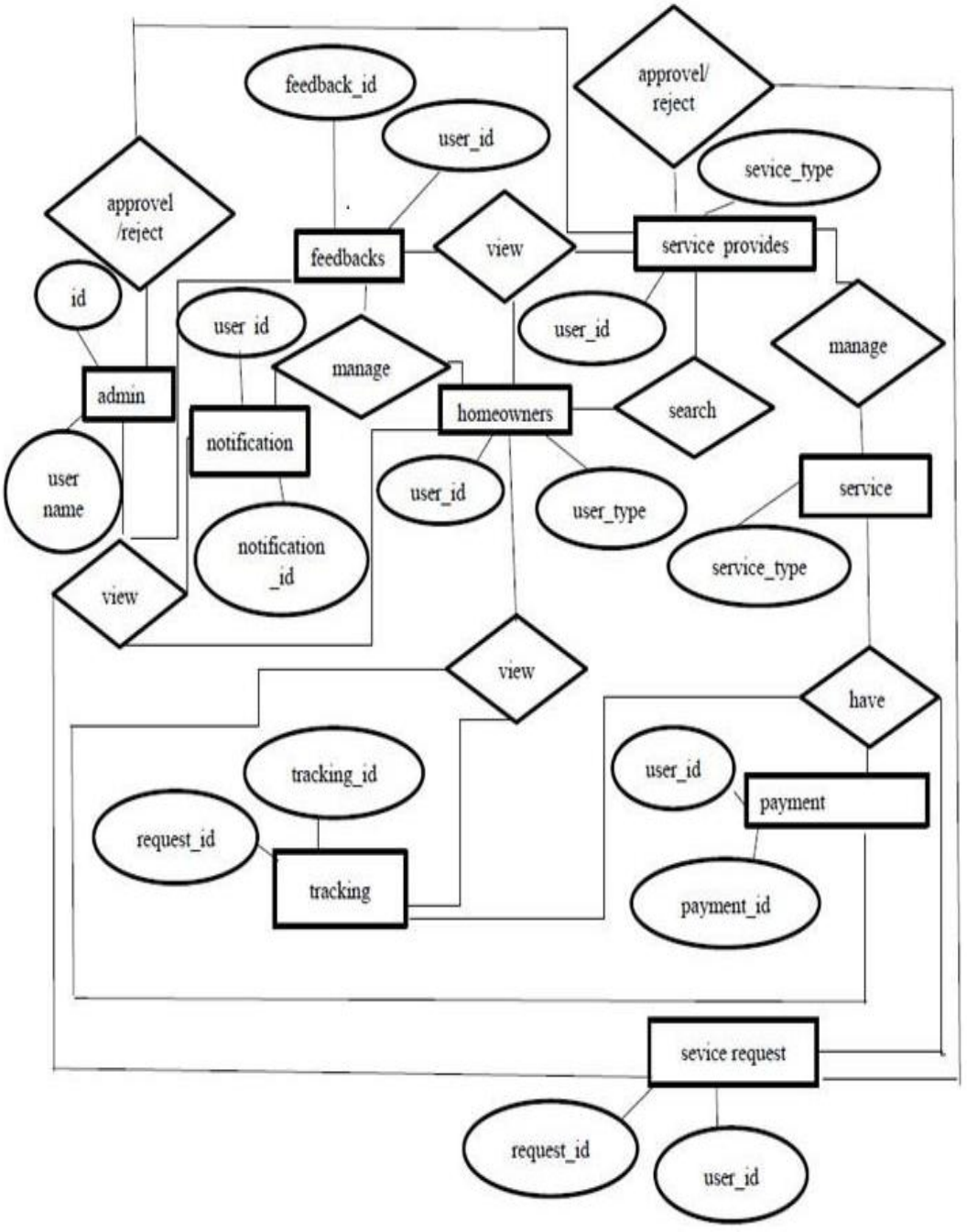


LEVEL 1 DFD SERVICE PROVIDER



(a) ENTITY RELATIONSHIP DIAGRAM

ER diagrams help in designing and understanding the structure of a database before it is built, making it easier to see how the data will be stored and linked together.



DESIGN DOCUMENT

(a) MODULES AND DESCRIPTION WITH EFFORT ESTIMATION

1. Admin Panel:
    - Manage Users (Homeowners/Tenants and Service Providers): Admins can register, approve, or reject user and service provider accounts, ensuring only verified and qualified professionals participate on the platform.
    - Manage Service Requests and Scheduling: Admins oversee all service requests, job assignments, and scheduling to ensure smooth operations and compliance with platform policies.
    - View and Manage Feedback: Admins can monitor and moderate feedback between users and service providers, maintaining service quality and resolving disputes when necessary.
    - Effort Estimation: 35 hours
2. Homeowner/Tenant Interface:
    - Profile Management: Homeowners and tenants can create and manage personal profiles, including property details and maintenance preferences.
    - Submit and Track Service Requests: Users can submit maintenance requests, view available service providers, and track the status of their requests in real-time.
    - Service Provider Search: Users can search for qualified service providers based on specific needs, such as plumbing, electrical, or general maintenance services.
    - Feedback Mechanism: After the completion of a job, users can leave reviews and ratings for service providers.
    - Effort Estimation: 45 hours
3. Service Provider Interface:
    - Profile Management: Service providers can create and manage their profiles, including uploading certifications, skills, and availability.
    - View and Accept Job Requests: Providers can browse service requests, view details, and accept or reject job offers based on their availability and expertise.
    - Job Scheduling and Management: Providers can manage upcoming jobs, update progress, and communicate directly with homeowners/tenants about the task.
    - Feedback Mechanism: Service providers can review and respond to feedback, helping to maintain service quality and improve their reputation.
    - Effort Estimation: 45 hours

Total Effort: 125 hours

(b) DATA STRUCTURES FOR ALL THE MODULES.

The **HomeFix Hub: A Home Maintenance Request Platform** leverages key data structures to efficiently manage and organize data across its three primary modules: Admin, Homeowner/Tenant, and Service Provider.

- In the **Admin Panel**, A hash table is utilized for quick retrieval and management of homeowner, tenant, and service provider records. This allows admins to efficiently approve or reject users by providing fast access to essential details. Additionally, a list structure is used to manage feedback entries from both users and service providers, enabling admins to easily iterate through reviews, address any issues, and make updates to profiles or the platform when necessary.
- For the **Homeowner/Tenant Interface**, User profiles are stored using a linked list, allowing dynamic updates to personal details and property information. This structure facilitates efficient modifications, such as adding or removing users from the system. A binary search tree (BST) is employed to organize available service providers, enabling users to quickly search and sort providers based on factors like service type, location, and availability, making it easier to find the right professional.
- In the **Service Provider Interface**, Job requests are managed using a queue, ensuring that service requests are handled in the order they are received, providing an orderly system for service providers to manage their tasks. Additionally, a stack is used to track recent feedback from users, allowing service providers to access the most recent reviews first and stay updated on their performance, making adjustments as needed to improve service quality.

(c) DATABASE DESIGN

Registration Table:

Field name	Data Type	Constraints	Description
user_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each user.
full_name	VARCHAR(100)	NOT NULL	User's full name.
email	VARCHAR(100)	NOT NULL, UNIQUE	User's email address.
phone_number	VARCHAR(15)	UNIQUE	User's phone number.
password	VARCHAR(255)	NOT NULL	User's password (hashed).
user_type	ENUM('admin', 'homeowner',	NOT NULL	Type of user: admin, homeowner, or
	'service_provider')		service provider.
property_address	VARCHAR(255)	NOT NULL	Address of the property for homeowners.
current_location	VARCHAR(255)	NOT NULL	Current location of the service provider.
years_of_experience	INT	NULL	Years of experience for service providers.
service_type	VARCHAR(50)	NOT NULL	Type of services offered by the service provider.
license_number	VARCHAR(50)	UNIQUE	Service provider's license number (if applicable).
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp of when the user was created.

Service RequestTable:

Field name	Data Type	Constraints	Description
request_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each maintenance request.
user_id	INT	FOREIGN KEY (user_id)	Identifier for the user who made the request.
service_provider_id	INT	FOREIGN KEY (service_provider_id)	Identifier for the assigned service provider.
request_description	TEXT	NOT NULL	Detailed description of the maintenance issue.
request_status	ENUM('pending', 'in_progress', 'completed')	NOT NULL	Current status of the maintenance request.
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp of when the request was created.
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Timestamp of the last update to the request.

Tracking Table:

Field name	Data Type	Constraints	Description
tracking_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each tracking entry.
request_id	INT	FOREIGN KEY (request_id)	Identifier for the associated service request.
service_provider_id	INT	FOREIGN KEY (service_provider_id)	Identifier for the service provider handling the request.
status		ENUM	NOT NULL	Current status of the service request.
location		VARCHAR(255)	NOT NULL	Current location of the service provider.
updated_at		DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Timestamp of the last update to the tracking information.

Feedback Table:

Field name	Data Type	Constraints	Description
feedback_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each feedback entry.
user_id	INT	FOREIGN KEY (user_id)	Identifier for the user giving feedback.
request_id	INT	FOREIGN KEY (request_id)	Identifier for the associated service request.
rating	INT	NOT NULL CHECK (rating BETWEEN 1 & 5)	Rating given by the user.
comments	TEXT	NULL	Additional comments or feedback from the user.
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp of when the feedback was submitted.

Payment table:

Field name	Data Type	Constraints	Description
payment_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each payment.
user_id	INT	FOREIGN KEY (user_id)	Identifier for the user making the payment.
request_id	INT	FOREIGN KEY (request_id)	Identifier for the associated service request.
amount	DECIMAL(10, 2)	NOT NULL	Amount paid for the service.
payment_method	ENUM('online', 'cash')	NOT NULL	Payment method used for the transaction.
payment_status	ENUM('pending', 'completed', 'failed')	NOT NULL	Current status of the payment.
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp of when the payment was made.

	Data Type	Constraints	Description
notification_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each notification.
user_id		FOREIGN KEY (user_id)	Identifier for the user receiving the notification.
message	TEXT	NOT NULL-	Notification message content.
is_read	BOOLEAN	DEFAULT FALSE	Indicates if the notification has been read.
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp of when the notification was created.

Data Integrity and Constraints

Data Integrity refers to the accuracy, consistency, and reliability of data stored in a database. It ensures that the data is correct and remains unaltered during operations like insertion, update, deletion, or retrieval. This is essential for maintaining the trustworthiness of the system and preventing errors or data corruption.

1.There are mainly four types:

Entity Integrity: Ensures each record has a unique identifier (Primary Key).

Referential Integrity: Maintains valid references between tables using Foreign Keys.

Domain Integrity: Ensures data entries are within valid ranges and formats.

User-Defined Integrity: Custom rules applied to maintain consistency according to business logic.

2. What are Constraints?

Constraints are rules enforced on database columns to ensure data integrity. They restrict the type of data that can be stored in a column.

3. Common Constraints Used in the Project

Based on the database design in your project (Home Maintenance Request Hub), here are key constraints used:

PRIMARY KEY: Uniquely identifies each record.
Example: user_id, request_id, feedback_id ensure each record is distinct.

FOREIGN KEY: Maintains relationships between tables.
Example: user_id in Feedback Table links to the Registration Table.

UNIQUE: Prevents duplicate values in a column.
Example: email and phone_number must be unique per user.

NOT NULL: Ensures a column cannot have a null (empty) value.
Example: full_name, password, request_description.

ENUM: Limits the values to a predefined list.
Example: user_type (admin, homeowner, service_provider), payment_method (online, cash).

CHECK: Validates data against a condition.
Example: rating INT CHECK (rating BETWEEN 1 AND 5) ensures only ratings 1–5 are allowed.

DEFAULT: Assigns a default value when none is specified.
Example: created_at DATETIME DEFAULT CURRENT_TIMESTAMP automatically records creation time.

4. Importance in Your Project

In the Home Maintenance Request Hub, maintaining data integrity ensures:
Reliable scheduling and tracking of service requests.
Accurate feedback and rating mechanisms.
Secure and valid user registrations.
Prevention of invalid or duplicate entries that could disrupt operations

(d) MODULES WITH PROCESS LOGIC

- **Admin Panel:** Admins manage the overall system, including the registration of both service providers and homeowners. They verify service provider credentials such as licenses and experience, approving or rejecting registration requests. Admins also oversee service requests, track activities, and monitor feedback from homeowners to ensure high service quality across the platform.

Homeowner Interface: Homeowners can register on the platform, create and manage their profiles, and submit maintenance requests. The system allows homeowners to describe their maintenance issues, select preferred service providers, and choose scheduling options. After the service is completed, they can provide feedback and rate the service providers, contributing to service improvement. The platform also enables real- time tracking of request statuses for a smooth experience.

Service Provider Interface: Service providers register by providing necessary credentials, such as license details and areas of expertise. Once approved by the admin, they can manage their profiles, view and accept maintenance requests, and update the status of their services (e.g., in progress or completed). Service providers can track payments for completed jobs and review feedback from homeowners, allowing them to enhance their service offerings. They can also communicate directly with homeowners for clarifications or updates regarding maintenance requests, fostering efficient interactions.

Procedural Design

Procedural Design refers to the detailed breakdown of how each feature or function of a software system will be executed step-by-step. It is like writing a manual for the software to follow when performing tasks, ensuring that logic flows smoothly and all interactions are handled efficiently. In your project, this concept is applied within each module and interaction, guiding how users, service providers, and the admin interact with the system.

1. Admin Panel Logic

Registration Management:
Accept input from a service provider or homeowner.
Validate data (e.g., email, license number, required fields).
If valid, mark as “pending approval.”
Admin logs in and reviews new registrations.
Admin approves or rejects the request.
Notification is sent to the user with the status.

Service Request Oversight:
Admin views all incoming requests.
Assigns or suggests service providers.
Monitors the current status (pending, in progress, completed).
Intervenes in disputes or escalated issues.

Feedback Monitoring:
Admin views feedback entries.
Flags inappropriate or abusive content.

2. Homeowner/Tenant Interface Logic
User Registration and Login:

Input validation for name, contact, and address.
Password encryption and storage.
On login, verify credentials and grant access.

Submitting a Service Request:
Select a service type (plumbing, electrical, etc.).

Tracking Requests:
Retrieve all requests submitted by the user.
Display request status (pending, scheduled, completed).
Allow editing or cancellation before confirmation.

Providing Feedback:
User selects a completed request.

3. Service Provider Interface Logic
Profile Management:
Register with full details, upload certification documents.
Choose services offered and available times.
Update skills or availability anytime.

Accepting Requests:
System shows available job requests.
Provider can click to view details and accept/reject.
Once accepted, status is updated and notification is sent to the homeowner.

Updating Job Progress:
Mark job as “In Progress” when started.
Add notes or comments if needed.
On completion, mark job as “Completed.”

Viewing Feedback:
Retrieve feedback tied to previous jobs.
Analyze ratings and comments.
Respond if allowed (to maintain transparency and accountability).

4. Common Procedural Elements Across Modules

Notification System:

Triggered during key events like registration approval, job updates, or payment confirmation.

Each user has a queue of notifications (read/unread status tracked).

Payment Processing:

Triggered after job completion.

User chooses payment method (online/cash).

Record transaction with status.

Generate receipt and update payment report.

Benefits of This Procedural Design:

Clarity and Organization: Each action follows a logical sequence.

Reusability: Procedures can be reused across similar modules.

Ease of Maintenance: Bugs are easier to locate and fix due to well-defined flows.

User-Focused Experience: Every step is crafted to optimize user interaction and satisfaction.

User Interface Design

User Interface (UI) Design refers to the process of designing the visual layout and interactive elements of a software application or website to ensure it is easy to use, intuitive, and visually appealing for users.

Key Concepts of User Interface Design:

1. User-Centered Design:

Focuses on the needs, preferences, and behaviors of the end-users.

Ensures the interface helps users achieve their goals with minimal effort.

2. Layout and Navigation:

Organizes content in a logical and visually clean way.

Includes elements like menus, buttons, tabs, and links for easy navigation.

3. Consistency:

Uses consistent colors, fonts, icons, and layout patterns to create familiarity and reduce the learning curve.

4. Responsiveness:
Adapts the interface to different devices and screen sizes (mobile, tablet, desktop).

5. Accessibility:
Designs interfaces that are usable by people with disabilities (e.g., using readable fonts, alt text for images, keyboard navigation).

6. Feedback Mechanisms:
Provides visual or audio cues when users perform actions (e.g., confirmation messages, loading spinners, error alerts).

7. Simplicity and Clarity:
Avoids clutter and presents only relevant information at each step to prevent user confusion.

Program Code:

Settings.py:

```
  INSTALLED_APPS = [
  'django.contrib.admin',
  'django.contrib.auth',
  'django.contrib.contenttypes',
  'django.contrib.sessions',
  'django.contrib.messages',
  'django.contrib.staticfiles',
  'users.apps.UsersConfig',
  'services',
  'ratings',
  'chat',
  'dashboard.apps.DashboardConfig',
  'crispy_forms',
  'crispy_bootstrap5',
  'widget_tweaks',
]

MIDDLEWARE = [
  'django.middleware.security.SecurityMiddleware',
  'django.contrib.sessions.middleware.SessionMiddleware',
```

```
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'homefix.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'homefix.wsgi.application'

# Database
# https://docs.djangoproject.com/en/5.1/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Views.py:

```
def home(request):
    context = {}

    if request.user.is_authenticated:
        print(f"User: {request.user}, Type: {request.user.user_type}")  # Debug user info

        if request.user.user_type == 'tenant':
            # Get recent service requests for tenant
            recent_requests = ServiceRequest.objects.filter(
                tenant=request.user
            ).order_by('-created_at')[:4]
```

```
    context['recent_requests'] = recent_requests

   # Get direct service requests made by the tenant
    tenant_direct_requests = DirectServiceRequest.objects.filter(
       tenant=request.user
    ).order_by('-created_at')[:4]

    print(f"Direct Requests for {request.user}: {tenant_direct_requests}")  # Debug output
    context['tenant_direct_requests'] = tenant_direct_requests

 elif request.user.user_type == 'serviceprovider':
    # Get upcoming jobs for service provider
    upcoming_jobs = ServiceRequest.objects.filter(
       offers__provider=request.user,
       offers__status='accepted',
       status__in=['assigned', 'in_progress']
    ).order_by('scheduled_date')[:4]
    context['upcoming_jobs'] = upcoming_jobs

    # Get direct service requests assigned to the provider
    direct_requests = DirectServiceRequest.objects.filter(
       provider=request.user
    ).order_by('-created_at')[:4]
    context['direct_requests'] = direct_requests

 return render(request, 'users/home.html', context)
 def register_tenant(request):
 if request.method == 'POST':
 form = TenantRegistrationForm(request.POST)
 if form.is_valid():
    user = form.save(commit=False)
    user.is_active = False
    user.save()
    send_verification_email(request, user)
    messages.success(request, 'Registration successful. Please check your email to verify your
    account.')
    return redirect('login')
   else:
 form = TenantRegistrationForm()
 return render(request, 'users/register_tenant.html', {'form': form})


 def register_service_provider(request):
  if request.method == 'POST':
 form = ServiceProviderRegistrationForm(request.POST)
 if form.is_valid():
    user = form.save(commit=False)
    user.is_active = False
    user.save()
```

```
        # Create provider profile
            provider_profile = ProviderProfile.objects.create(
                user=user,
                rating=0.0,
                total_reviews=0,
                is_available=True,
                availability={}
            )
            # Add selected categories
            categories = form.cleaned_data.get('categories', [])
            provider_profile.categories.set(categories)

            send_verification_email(request, user)
            messages.success(request, 'Registration successful. Please check your email to verify your
          account.')
            return redirect('login')
    else:
        form = ServiceProviderRegistrationForm()
    return render(request, 'users/register_service_provider.html', {'form': form})
```

urls.py:

```
from django.urls import path
from django.contrib.auth import views as auth_views
from . import views
from .forms import CustomPasswordResetForm

urlpatterns = [
    path('', views.home, name='home'),
    path('register/tenant/', views.register_tenant, name='register_tenant'),
    path('register/service-provider/', views.register_service_provider,
name='register_service_provider'),
    path('login/', auth_views.LoginView.as_view(template_name='users/login.html'), name='login'),
    path('logout/', views.user_logout, name='logout'),
]
```

Models.py:

```
class CustomUser(AbstractUser):
    USER_TYPE_CHOICES = (
        ('tenant', 'Tenant'),
        ('serviceprovider', 'Service Provider'),
        ('admin', 'Admin'),
    )

    user_type = models.CharField(max_length=15, choices=USER_TYPE_CHOICES)
    email = models.EmailField(unique=True)
    phone_regex = RegexValidator(
        regex=r'^[6-9]\d{9}$',
        message="Phone number must start with 6,7,8, or 9 and be exactly 10 digits long."
    )
```

```
phone_number = models.CharField(validators=[phone_regex], max_length=10)

 is_email_verified = models.BooleanField(default=False)
is_approved = models.BooleanField(default=False)

def __str__(self):
    return self.username
class TenantProfile(models.Model):
user = models.OneToOneField(
    "CustomUser", on_delete=models.CASCADE, related_name="tenant_profile"
)
location = models.CharField(max_length=255)
phone_number = models.CharField(max_length=10, blank=True, null=True)
move_in_date = models.DateField(blank=True, null=True)
maintenance_preferences = models.TextField(blank=True)  # Flexible storage

def __str__(self):
    return f"{self.user.username}'s Tenant Profile"

class Meta:
    verbose_name = "Tenant Profile"
    verbose_name_plural = "Tenant Profiles"

class ServiceProviderProfile(models.Model):
user = models.OneToOneField("users.CustomUser", on_delete=models.CASCADE,
related_name='provider_profile')
service_location = models.CharField(max_length=255)
service_provided = models.ManyToManyField("services.ServiceCategory",
related_name='service_provided')
certifications = models.TextField(blank=True)

def __str__(self):
    return f"{self.user.username}'s Service Provider Profile"
```

forms.py:

```
class TenantRegistrationForm(UserCreationForm):

class Meta:
    model = CustomUser
    fields = ('username', 'email', 'phone_number', 'password1', 'password2')

def save(self, commit=True):
    user = super().save(commit=False)
    user.user_type = 'tenant'
    if commit:
        user.save()
        TenantProfile.objects.create(
            user=user
        )
```

```
    return user

class ServiceProviderRegistrationForm(UserCreationForm):
service_location = forms.CharField(max_length=255)
categories = forms.ModelMultipleChoiceField(
    queryset=ServiceCategory.objects.all(),
    widget=forms.CheckboxSelectMultiple,  # Converts multi-select into checkboxes
    required=True,
    help_text="Select the service categories you provide"
)
certifications = forms.CharField(widget=forms.Textarea, required=False)

class Meta:
    model = CustomUser
    fields = ('username', 'email', 'phone_number', 'password1', 'password2')

def save(self, commit=True):
    user = super().save(commit=False)
    user.user_type = 'serviceprovider'
    if commit:
        user.save()
        ServiceProviderProfile.objects.create(
            user=user
        )
    return user
```

templates:

register.html:

```
{% extends 'users/base.html' %}

{% block title %}Register as Tenant{% endblock %}

{% block content %}
<div class="max-w-md mx-auto">
  <div class="bg-white rounded-lg shadow-sm p-8">
    <h2 class="text-2xl font-bold text-center text-gray-900 mb-8">Register as Tenant</h2>

    <form method="post" novalidate class="space-y-6">
      {% csrf_token %}
      {% for field in form %}
        <div>
        <label for="{{ field.id_for_label }}" class="block text-sm font-medium text-gray-700 mb-1">
            {{ field.label }}
          </label>

          {% if field.name == 'password1' or field.name == 'password2' %}
            <input
              type="{{ field.field.widget.input_type }}"
              name="{{ field.name }}"
```

```
    id="{{ field.id_for_label }}"
                    required
                    class="appearance-none block w-full px-3 py-2 border {% if field.errors %}border-red-
    300{% else %}border-gray-300{% endif %} rounded-md shadow-sm placeholder-gray-400
    focus:outline-none focus:ring-indigo-500 focus:border-indigo-500 sm:text-sm"
                  >
                {% else %}
                  <input
                    type="text"
                    name="{{ field.name }}"
                    id="{{ field.id_for_label }}"
                    value="{{ field.value|default:'' }}"
                    required
                    class="appearance-none block w-full px-3 py-2 border {% if field.errors %}border-red-
   300{% else %}border-gray-300{% endif %} rounded-md shadow-sm placeholder-gray-400
    focus:outline-none focus:ring-indigo-500 focus:border-indigo-500 sm:text-sm"
                  >
                {% endif %}

                {% if field.errors %}
                  <p class="mt-1 text-sm text-red-600">{{ field.errors|join:", " }}</p>
                {% endif %}

                {% if field.help_text %}
                  <p class="mt-1 text-sm text-gray-500">{{ field.help_text }}</p>
                {% endif %}
              </div>
            {% endfor %}

            <button type="submit" class="w-full flex justify-center py-2 px-4 border border-transparent
   rounded-md shadow-sm text-sm font-medium text-white bg-indigo-600 hover:bg-indigo-700
   focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-indigo-500">
              Register
            </button>
          </form>

          <div class="mt-6 text-center">
            <p class="text-sm text-gray-600">
              Already have an account?
              <a href="{% url 'login' %}" class="font-medium text-indigo-600 hover:text-indigo-500">
                Login here
              </a>
            </p>
          </div>
        </div>
      </div>
    {% endblock %}
```

Login.html:

```
{% extends 'users/base.html' %}

{% block title %}Login{% endblock %}

      {% block content %}
     <div class="max-w-md mx-auto">
     <div class="bg-white rounded-lg shadow-sm p-8">
      <h2 class="text-2xl font-bold text-center text-gray-900 mb-8">Login to HomeFix</h2>

      <form method="post" novalidate class="space-y-6">
       {% csrf_token %}
        {% for field in form %}
        <div>
        <label for="{{ field.id_for_label }}" class="block text-sm font-medium text-gray-700 mb-1">
                  {{ field.label }}
               </label>
               {{ field.errors }}
               <input
                  type="{{ field.field.widget.input_type }}"
                  name="{{ field.name }}"
                  id="{{ field.id_for_label }}"
                  required
                  class="appearance-none block w-full px-3 py-2 border {% if field.errors %}border-red-
300{% else %}border-gray-300{% endif %} rounded-md shadow-sm placeholder-gray-400
focus:outline-none focus:ring-indigo-500 focus:border-indigo-500 sm:text-sm"
               >
               {% if field.errors %}
                  <p class="mt-1 text-sm text-red-600">{{ field.errors|join:", " }}</p>
               {% endif %}
            </div>
         {% endfor %}

         <button type="submit" class="w-full flex justify-center py-2 px-4 border border-transparent
rounded-md shadow-sm text-sm font-medium text-white bg-indigo-600 hover:bg-indigo-700
focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-indigo-500">
            Login
         </button>
      </form>

      <div class="mt-6 space-y-4">
         <div class="text-center">
            <a href="{% url 'password_reset' %}" class="text-sm font-medium text-indigo-600
hover:text-indigo-500">
               Forgot your password?
            </a>
         </div>
```

```
    <div class="relative">
      <div class="absolute inset-0 flex items-center">
        <div class="w-full border-t border-gray-300"></div>
      </div>
      <div class="relative flex justify-center text-sm">
        <span class="px-2 bg-white text-gray-500">Don't have an account?</span>
      </div>
    </div>

    <div class="grid grid-cols-2 gap-3">
      <a href="{% url 'register_tenant' %}" class="inline-flex justify-center items-center px-4 py-2
  border border-gray-300 shadow-sm text-sm font-medium rounded-md text-gray-700 bg-white
   hover:bg-gray-50 focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-indigo-500">
        <i class="fas fa-home mr-2"></i>
        Register as Tenant
      </a>
      <a href="{% url 'register_service_provider' %}" class="inline-flex justify-center items-center
   px-4 py-2 border border-gray-300 shadow-sm text-sm font-medium rounded-md text-gray-700
   bg-white hover:bg-gray-50 focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-indigo-
   500">
        <i class="fas fa-tools mr-2"></i>
        Register as Provider
      </a>
    </div>
  </div>
</div>
</div>
 {% endblock %}
```

TESTING DETAILS

1. **Unit Testing:** Unit testing focuses on individual components or modules of the HomeFix Hub system to ensure that each part functions correctly in isolation. Developers write test cases for specific functions or methods, such as user registration, service request submissions, and feedback processing, validating their behavior and identifying any issues early in the development process. For example, unit tests ensure that service requests are created with the correct details and that feedback is accurately recorded and associated with the respective service providers.
2. **Integration Testing:** Integration testing examines the interaction between multiple modules to verify that they work together as intended. This type of testing includes verifying the interaction between the service request module and the notification system, ensuring that notifications are sent to both homeowners and service providers upon request submission or status updates. Additionally, integration testing checks that the payment processing system correctly links to the service request module, facilitating smooth transactions and accurate billing

3. **System Testing:** System testing evaluates the entire integrated system to ensure it meets all specified requirements. This comprehensive testing validates all functionalities, such as user registration (for both homeowners and service providers), service request submissions, payment processing, and feedback mechanisms. It ensures that the system operates seamlessly from submitting a maintenance request to completing the service, including verifying that notifications, tracking features, and payment flows function correctly across the user interfaces for homeowners, service providers, and admins.

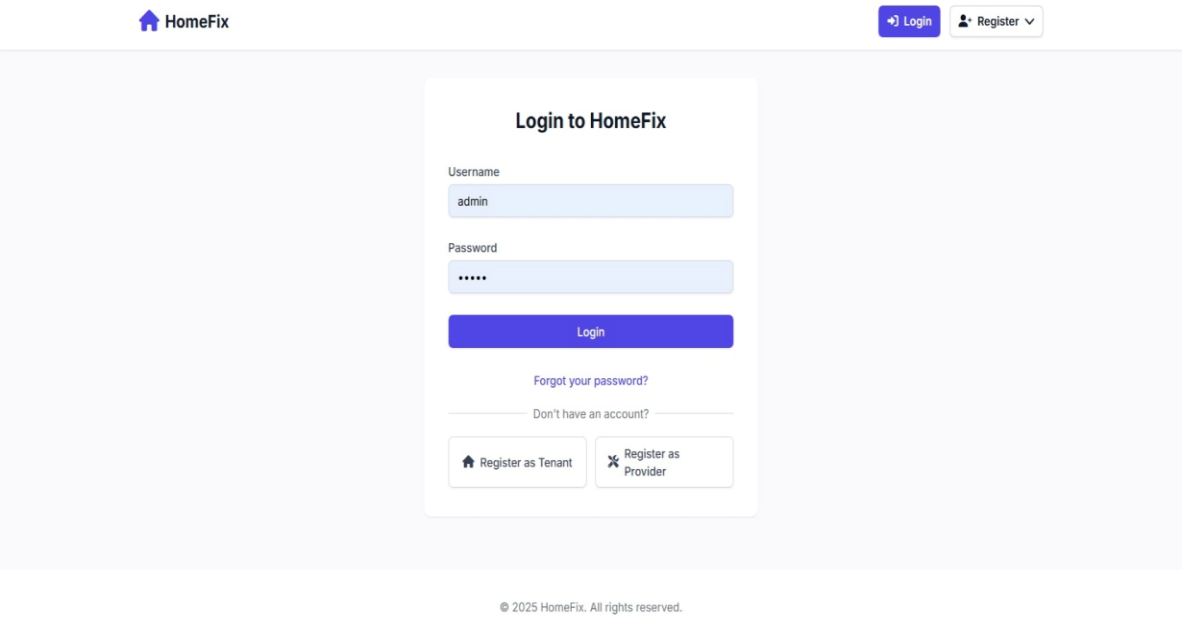
(e) REPORT GENERATION

- **Service Provider Performance Report:** Summarizes the performance metrics of service providers, including homeowner feedback, number of completed requests, average ratings, and response times. This report helps assess the efficiency, quality of service, and reliability provided by service providers on the platform.
- **Maintenance Request Report**: Details the number of maintenance requests submitted by homeowners, including completed, pending, and cancelled requests over a specified period. This report provides insights into service demand, utilization rates, and request trends, helping to optimize resource allocation and improve system performance.
- **Feedback Report**: Compiles feedback received from homeowners regarding their experiences with service providers, highlighting areas for improvement and overall satisfaction ratings. This report enables both service providers and administrators to identify patterns in user feedback and enhance the quality of service based on homeowner input.
- **Payment Report**: Generates a comprehensive overview of the payments processed through the platform, detailing transactions based on payment methods (online or cash), amounts paid, and the associated request IDs. The report also includes a breakdown of the total revenue generated over a specific period, categorized by service providers and homeowners.

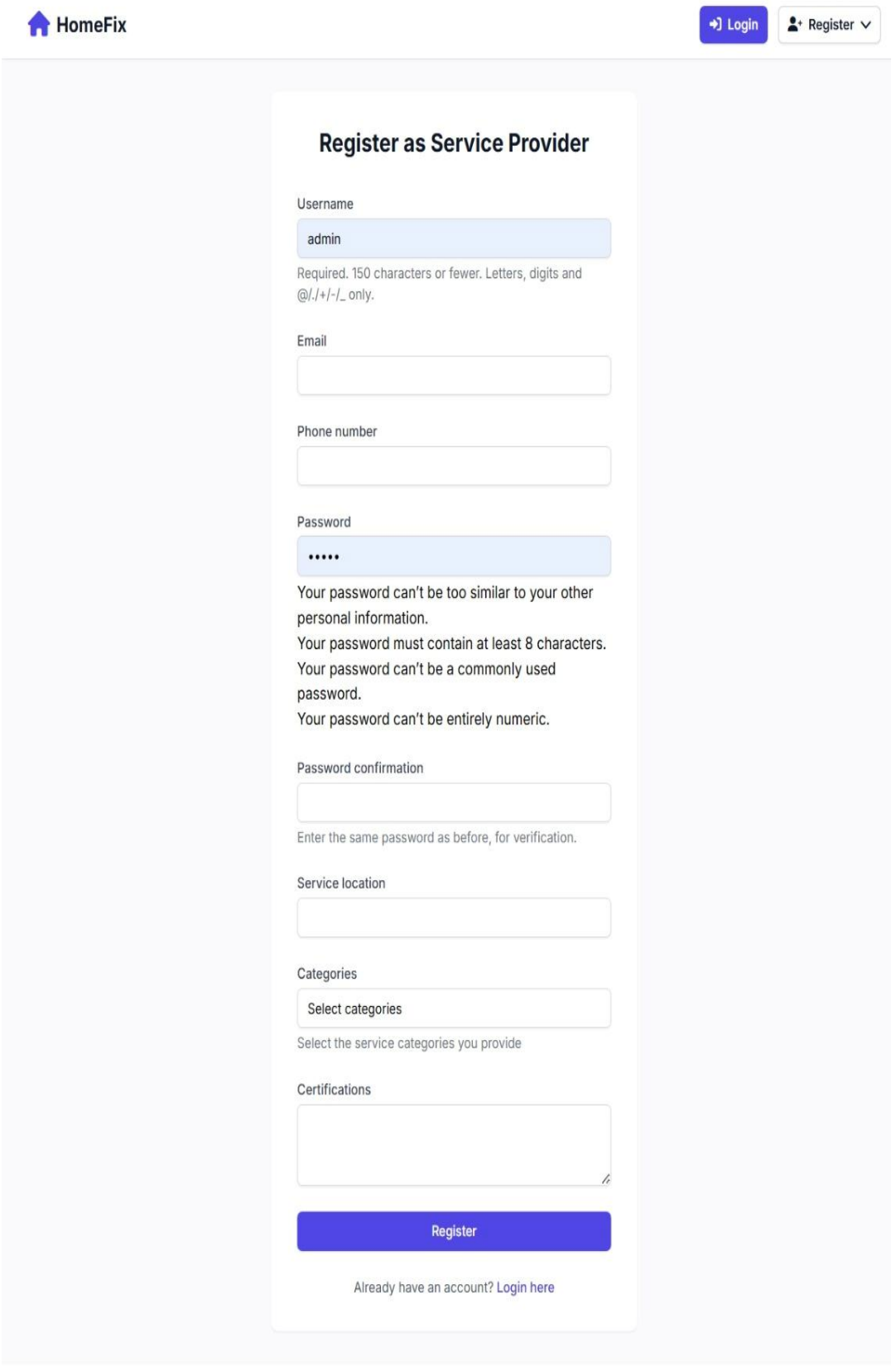
INPUT AND OUTPUT SCREENS

INPUT SCREENSHOTS

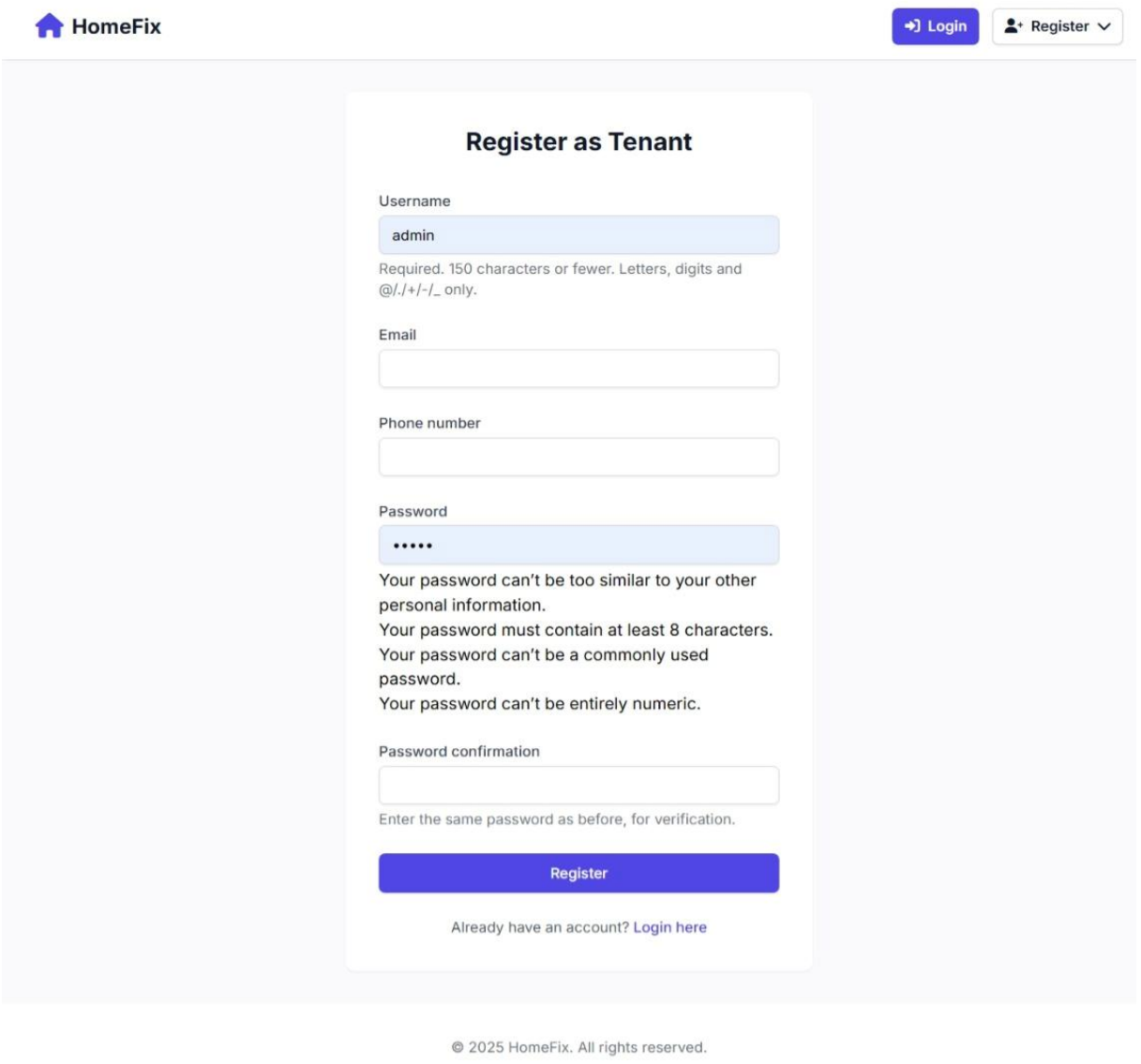
Login



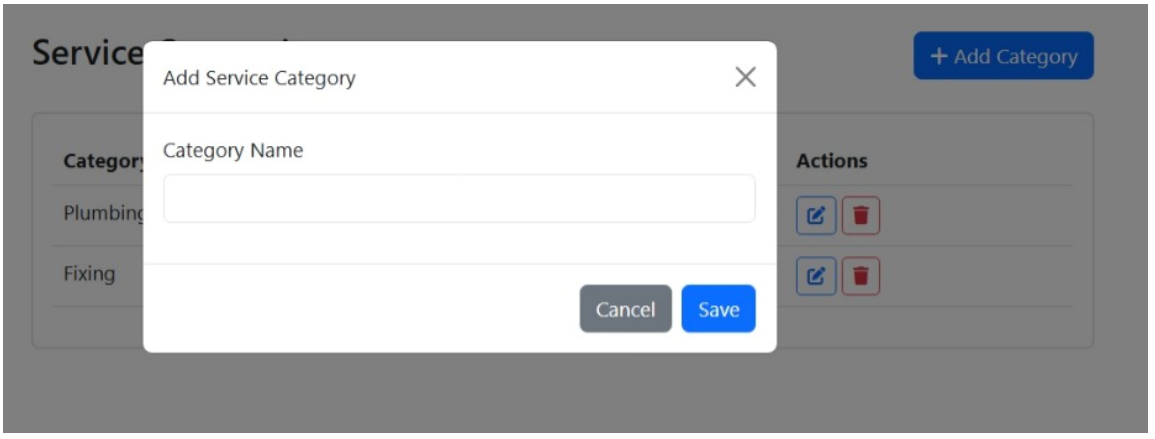
Provider Register



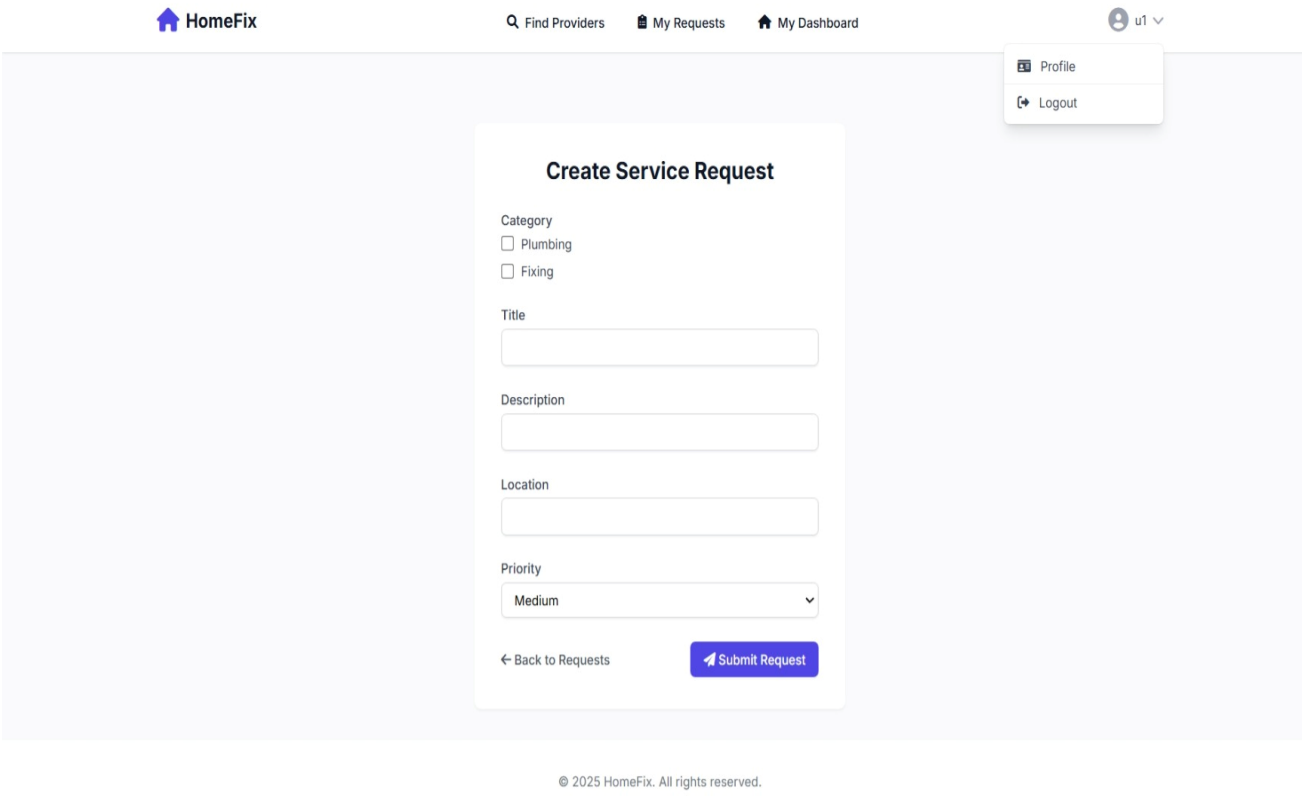
Tenant Register



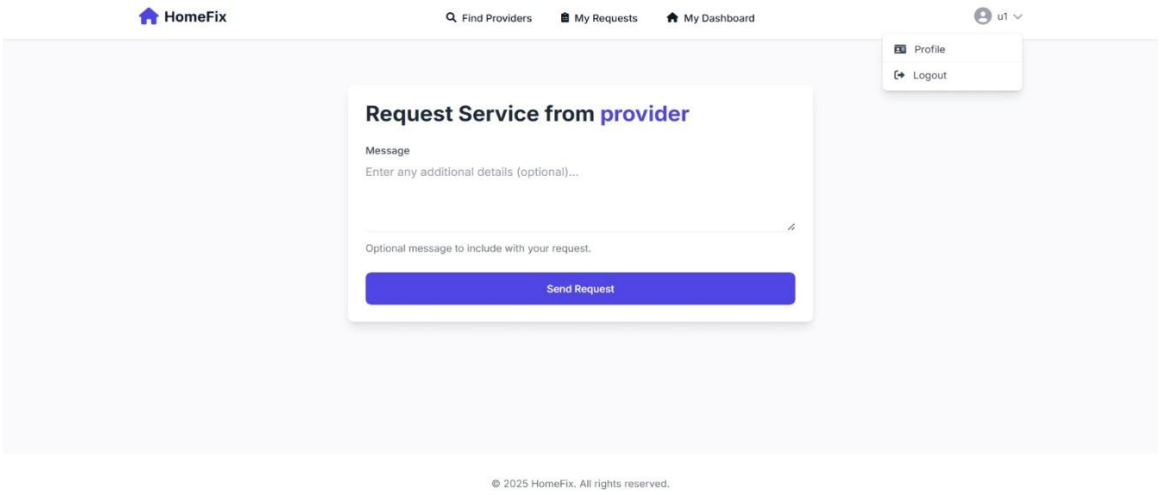
Add Category



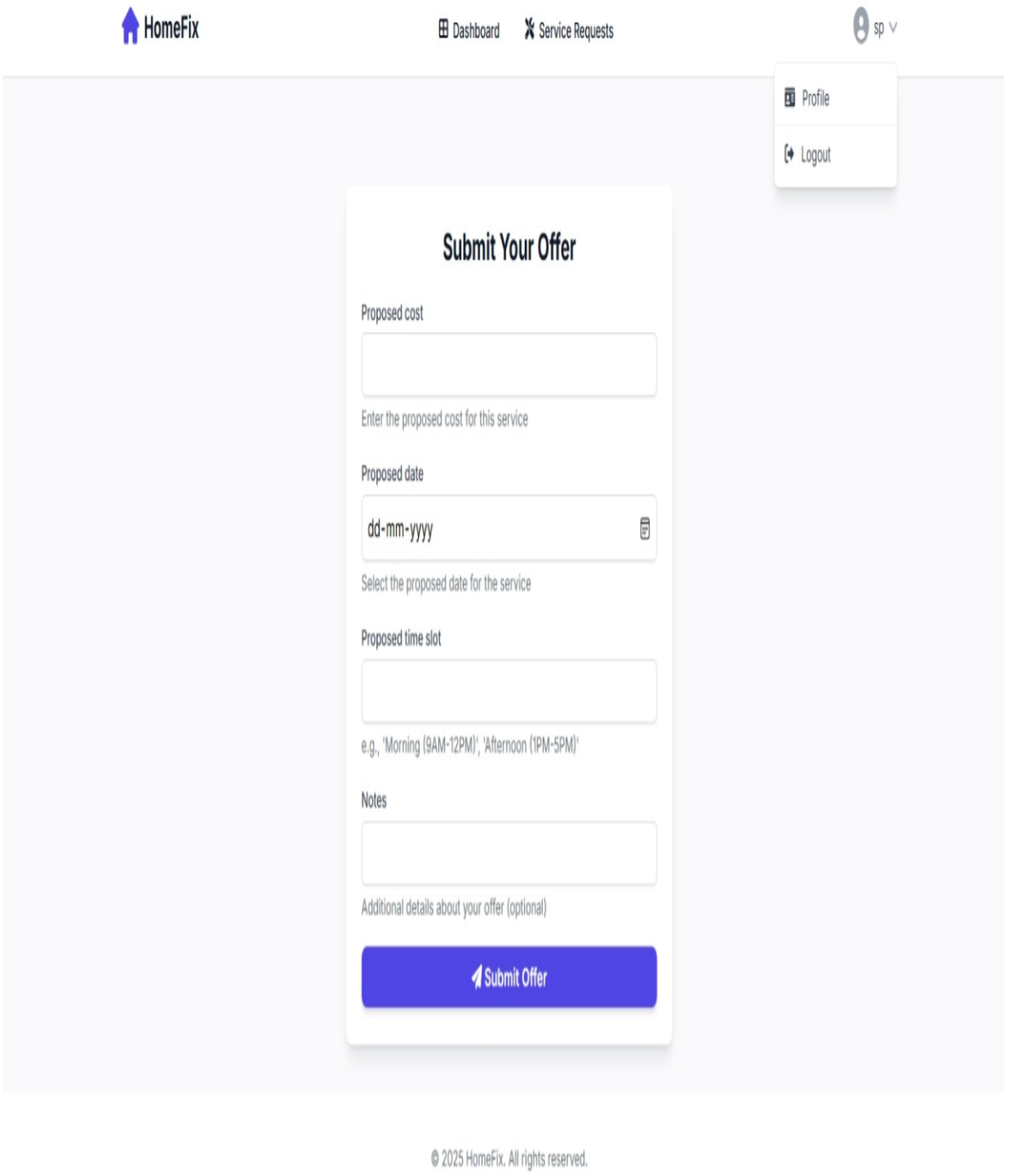
Request Serivce



Send Request Message

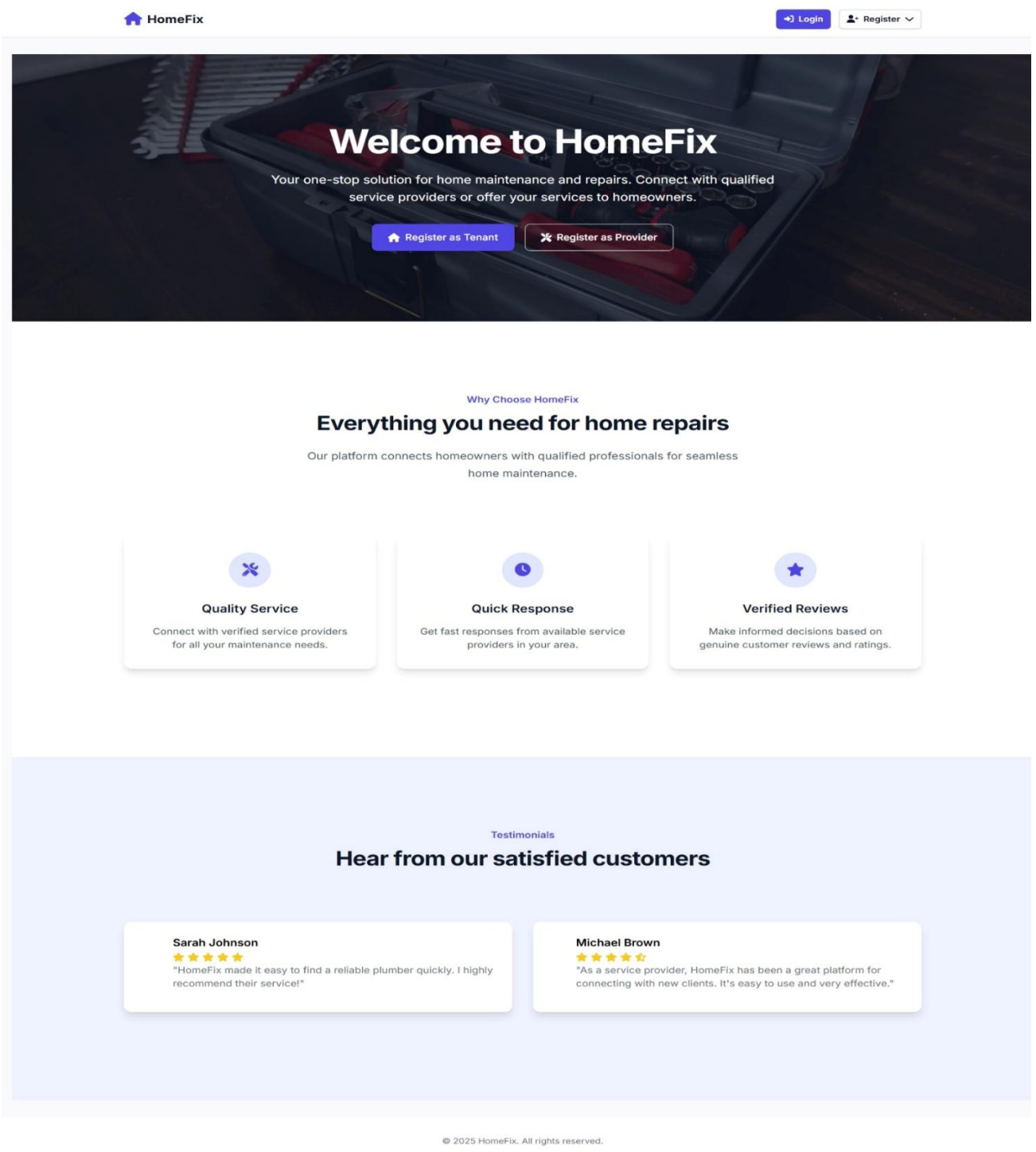


Submit Offer

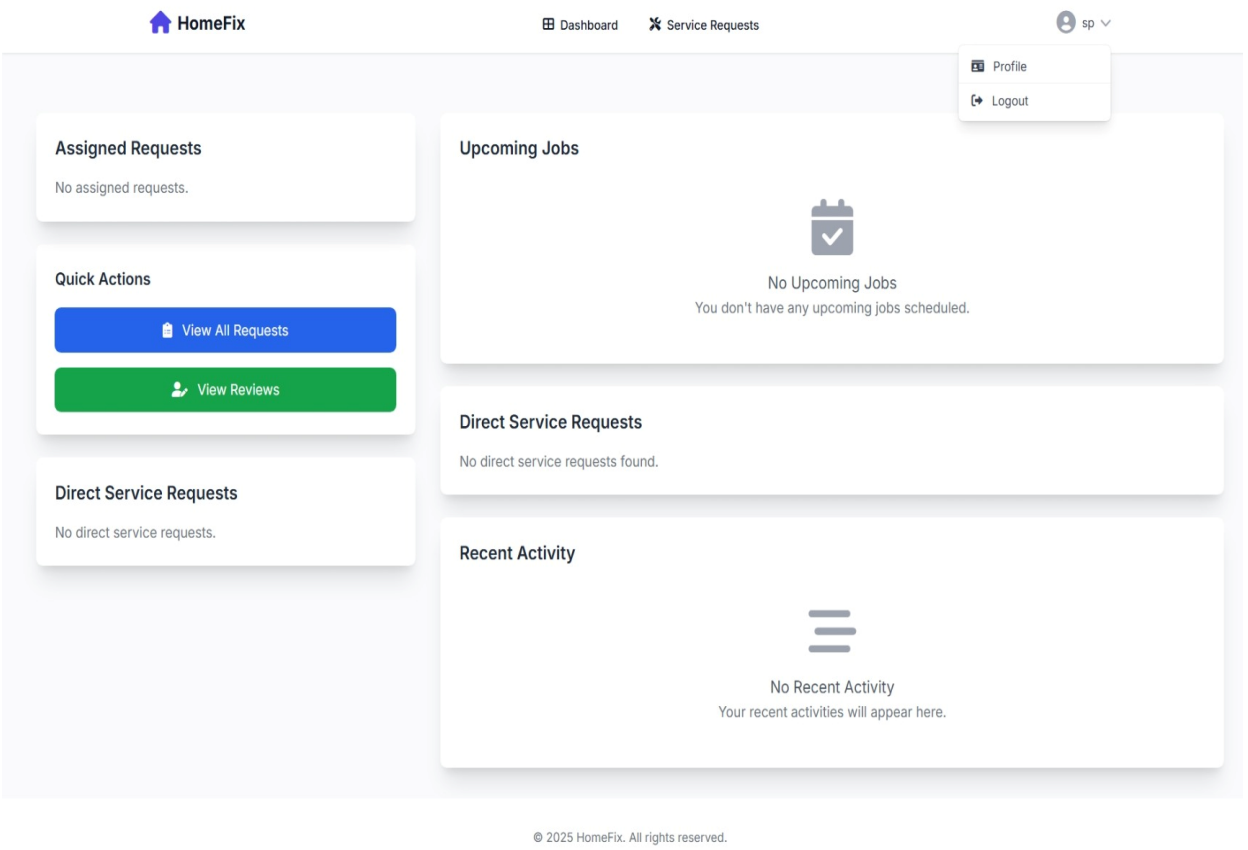


Output Screenshots

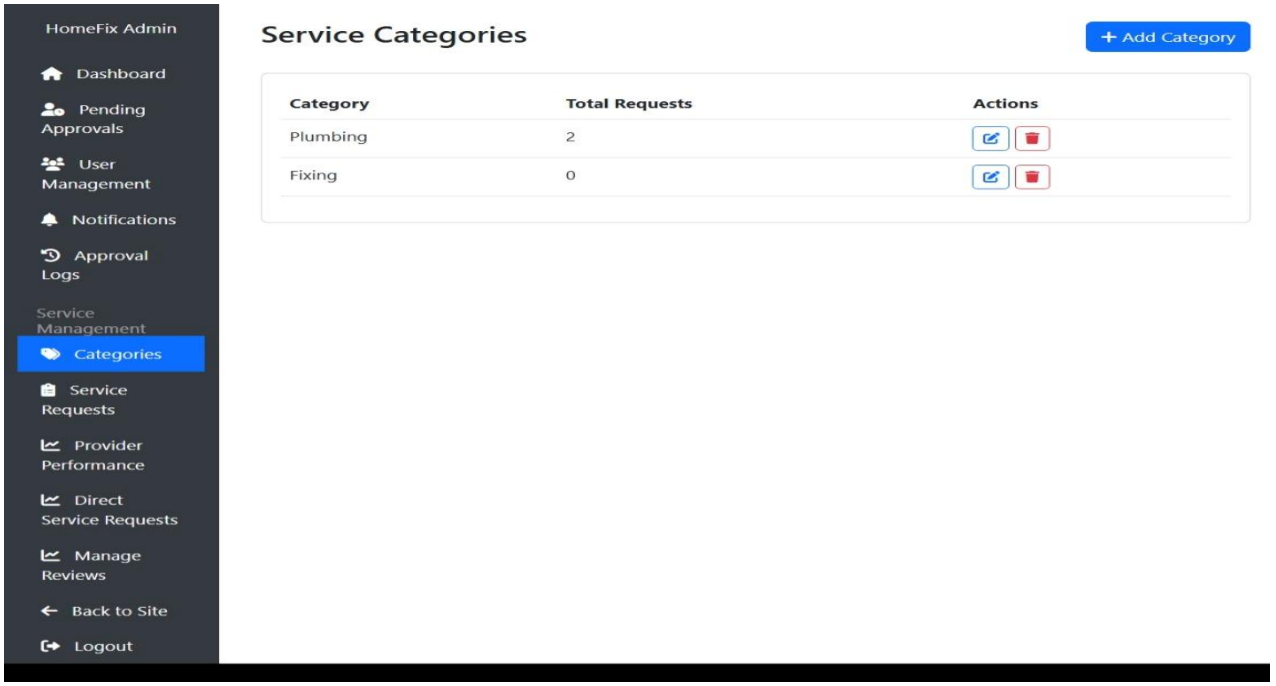
Admin View User :



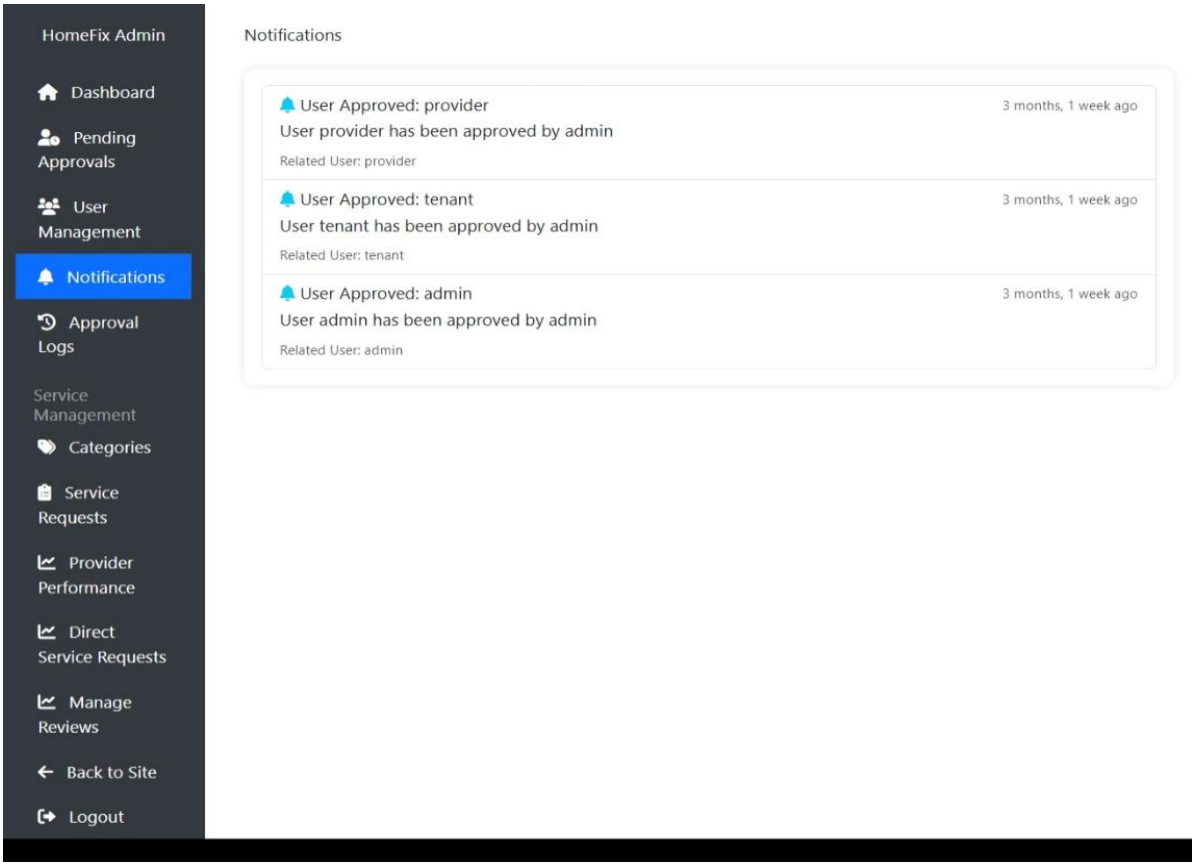
Service Request Details



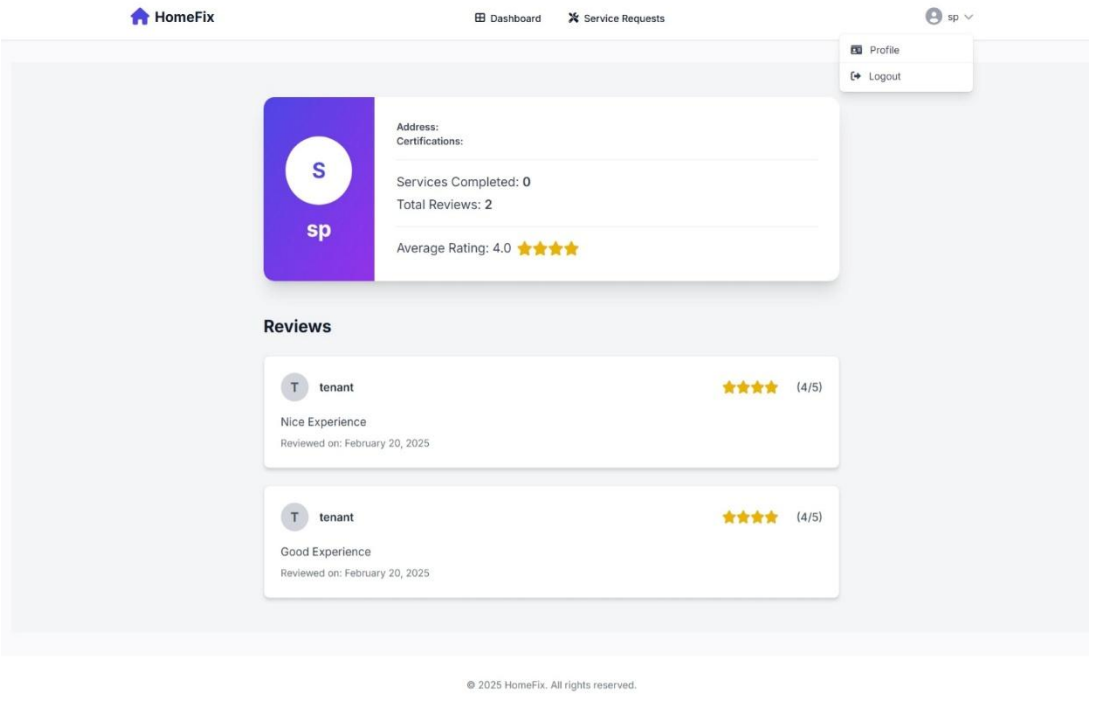
View Category



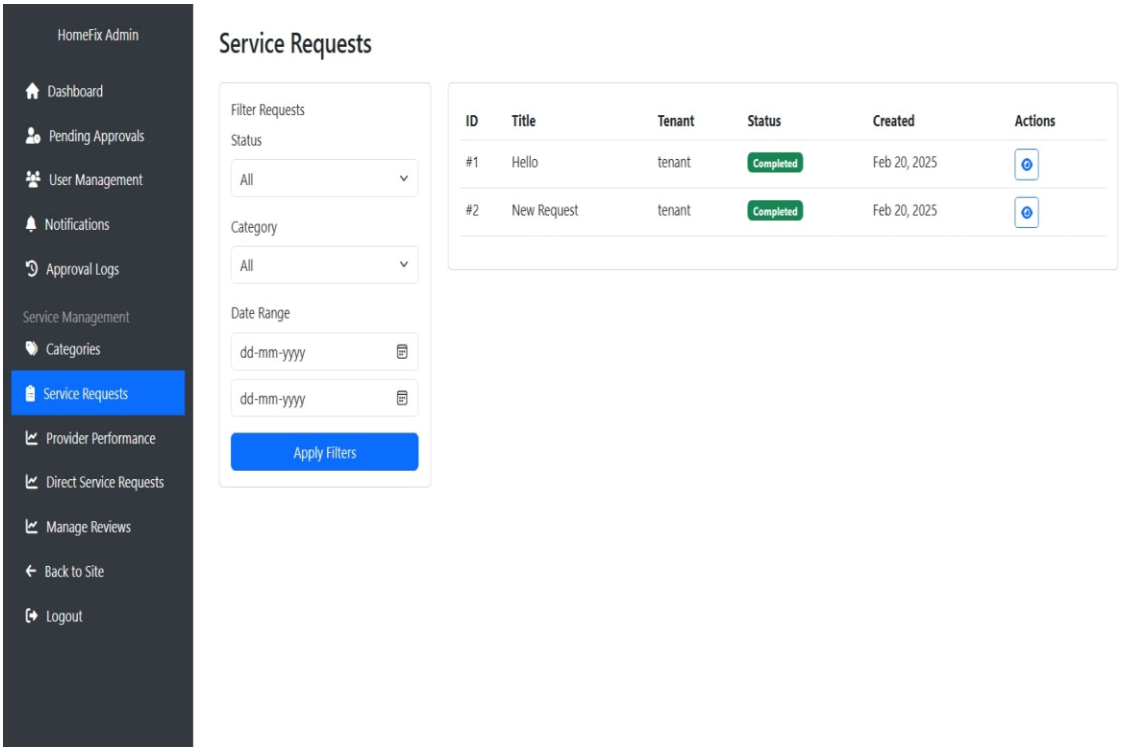
View Notification



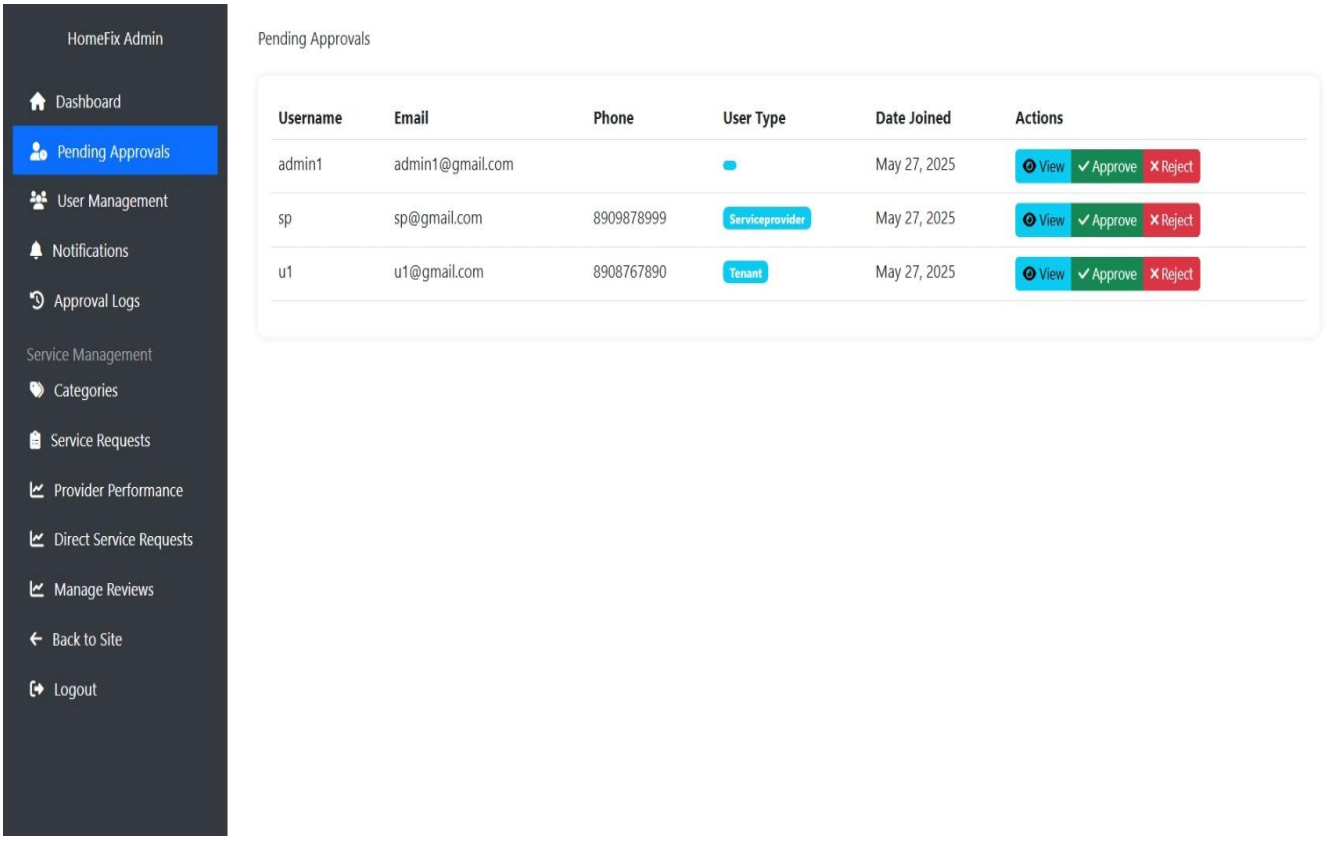
View Reviews



View Service Request



View Users



IMPLEMENATION OF SECURITY

¶ The implementation of security in the Home Maintenance Request Hub is crucialto protect sensitive data, ensure trustworthy interactions between users and service providers, and maintain the platform's integrity. The project incorporates a multi-layered security approach, covering user authentication, data protection, and secure communication. Below is a detailed expansion of how security is implemented across the system:

1. User Authentication and Authorization

Secure Login System: The platform enforces user authentication through a secure login mechanism where credentials (email and password) are verified.

Password Hashing: User passwords are stored in the database using secure hashing algorithms (e.g., SHA-256 or bcrypt), ensuring they cannot be easily retrieved or compromised.

Role-Based Access Control (RBAC): Different access levels are assigned to admins, service providers, and homeowners/tenants. This ensures that user conly access functions and data relevant to their roles.

2. Data Validation and Sanitization

Input Validation: All user inputs are validated both on the client and server sides to prevent SQL injection, cross-site scripting (XSS), and other injection attacks.

Data Sanitization: Inputs such as feedback, comments, and addresses are sanitized to prevent malicious script embedding.

3. Database Security

Secure Queries: Prepared statements and parameterized queries are used for database access to prevent SQL injection.

Access Restrictions: Database access is limited to authenticated backend services. Direct database access from the front-end is strictly prohibited.

Encryption of Sensitive Data: Critical data such as payment details are encrypted using secure encryption protocols before storage or transmission.

4. Secure Communications

HTTPS Protocol: All data transmitted between users and the server is encrypted using HTTPS to prevent eavesdropping and data tampering.

Token-Based Authentication: JSON Web Tokens (JWT) or session tokens are used for maintaining secure sessions during user interactions.

5. Admin Verification

Manual Credential Verification: Admins are responsible for verifying service provider credentials such as licenses and experience before approving their accounts.

Activity Monitoring: Admins monitor platform activities for unusual behavior, spam, fraudulent requests and have the authority to take corrective actions.

6. Payment Security

Secure Payment Gateways: All online transactions are processed via secure and PCI-compliant third-party payment gateways.

Transaction Logs: All payments are logged and monitored for anomalies or failures. Paymen statuses are updated in real time to inform users of successful transactions.

7. Notification and Alerts

Security Alerts: Users are notified via email or in-platform notifications about Important activities such as password changes, new logins, and completed transactions.

Two-Factor Authentication (Future Enhancement): Planned integration of 2FA to provide an additional layer of security during user login.

8. Audit and Logging

Activity Logs: Every critical user action is logged (e.g., login attempts, request updates, feedback submissions) for audit and traceability.

Error and Exception Handling: Robust error-handling mechanisms prevent security leakage through debug information in production.

These security implementations ensure that the Home Maintenance Request Hub provides a safe, trustworthy, and reliable environment for homeowners, tenants, service providers, and administrators. Security remains a core focus throughout the software development lifecycle, with ongoing enhancements based on evolving threats and user needs.

LIMITATIONS

Despite offering a robust and functional platform for managing home maintenance services, the Home Maintenance Request Hub system has certain limitations that could affect its scalability, user experience, and operational efficiency:

1. Platform Dependency
The system is currently developed as a web-based platform and may not provide a seamless experience on mobile devices. Without a dedicated mobile app, users may face difficulties accessing full functionality on smaller screens or when internet connectivity is limited.

2. Limited Automation
While service requests and scheduling are streamlined, the platform lacks advanced automation features such as AI-based job matching, dynamic rescheduling, or predictive maintenance alerts. This can lead to manual overhead in assigning tasks and handling reschedules.

3. Geographical Restrictions
The system assumes a localized service area and does not currently account for regional differences in licensing, availability, or pricing of service providers. Expansion to a broader geographical area would require significant customization.

4. Scalability Constraints
The current architecture may face challenges when scaling to support a large user base or high volume of concurrent requests, especially without cloud-based infrastructure or load balancing mechanisms.

5. Security and Privacy Considerations
Although basic security measures like password hashing are implemented, there may be a lack of advanced features such as multi-factor authentication, encrypted communication, or compliance with data protection regulations (e.g., GDPR), which could expose the platform to security risks.

6. Limited Payment Integration
The platform supports only basic payment methods (online or cash). It lacks integration with popular payment gateways, digital wallets, or invoicing systems, which may limit user convenience and flexibility in transactions.

7. Manual Verification of Service Providers
The current process relies heavily on admin review for validating service provider credentials. This manual verification process can be time-consuming, error-prone, and may not scale well with an increasing number of providers.

8. Absence of Real-Time Communication Tools
There is no integrated real-time chat or video communication feature, which can hinder immediate interaction between service providers and homeowners, especially in cases requiring urgent clarification or coordination.

9. Limited Analytics and Reporting
Although some basic reports are available, the system lacks a comprehensive analytics dashboard that can provide deep insights into user behavior, operational efficiency, or service trends.

10. Dependence on Internet Connectivity
As a fully web-based solution, the system requires continuous internet access for all operations. Users in areas with poor or intermittent connectivity may experience service disruptions or limitations in functionality.

FUTURE SCOPE AND FURTHER ENHANCEMENT OF THE PROJECT

1. Mobile Application Development: Expanding the platform into a mobile application to provide users with convenient access to home maintenance services and real-time notifications regarding service requests and updates.

2. AI-Based Maintenance Scheduling: Implementing an AI-driven scheduling system that predicts maintenance needs based on user history and seasonal requirements, ensuring timely service provision.

3. Multi-Service Provider Integration: Collaborating with a wider range of service providers, including specialized technicians, to enhance the variety and quality of home maintenance services available to users.

4. Customer Feedback System: Establishing a robust feedback system to collect user reviews and ratings, which can help improve service quality and provider accountability.

5. Subscription and Membership Plans: Introducing subscription-based plans that offer benefits such as discounted services, priority booking, and regular maintenance check-ups for loyal customers.

6. Smart Home Integration: Exploring integration with smart home devices to provide proactive maintenance alerts and remote troubleshooting services, enhancing the overall user experience.

7. Data Analytics for Service Improvement: Utilizing data analytics to track service usage patterns, customer preferences, and feedback, enabling continuous improvement of service offerings and user satisfaction.

Bibliography

1.Sommerville, Ian. Software Engineering. 10th ed., Pearson, 2015.

2. Pressman, Roger S. Software Engineering: A Practitioner's Approach. 8th ed., McGraw-Hill Education ,2014.

3. Welling, Luke, and Laura Thomson. PHP and MySQL Web Development. 5th ed., Addison-Wesley Professional, 2016.

4. Holtsnider, Bill, and Brian D. Jaffe. IT Manager's Handbook: Getting Your New Job Done. 3rd ed., Morgan Kaufmann, 2012.

5. Fowler, Martin. UML Distilled: A Brief Guide to the Standard Object Modeling Language. 3rd ed., Addison-Wesley, 2004.

6. Beizer, Boris. Software Testing Techniques. 2nd ed., Dreamtech Press, 2002.

7. Django Software Foundation. Django Documentation. Available at: https://docs.djangoproject.com/

8. Mozilla Developer Network. HTML, CSS, and JavaScript Documentation. Available at: https://developer.mozilla.org/

9. Elmasri, Ramez, and Shamkant B. Navathe. Fundamentals of Database Systems. 7th ed., Pearson, 2015.

10. Preece, Jenny, Yvonne Rogers, and Helen Sharp. Interaction Design: Beyond Human-Computer Interaction. 4th ed., Wiley, 2015.